

Amlogic

NNIDE 工具介绍

Revision: 1.0.0

Release Date: 2024-07-01

Copyright

© 2024 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

更改记录

版本 1.0.0 (2024-07-01)

第一版。

Confidential!
nick@khadas.com
2024-11-20 08:49:14

目录

更改记录.....	iii
1. NNIDE 简介	1
1.1 NNIDE_tool 目录简介	1
2. NNIDE 功能	2
2.1 API 接口介绍.....	2
2.1.1 amlnn = AMLNN()	2
2.1.2 amlnn.init().....	2
2.1.3 amlnn.set_input().....	2
2.1.4 amlnn.inference()	2
2.1.5 amlnn.destroy()	3
2.1.6 amlnn.similarity_and_distance()	3
2.2 模型推理.....	3
2.3 精度对比.....	3
2.4 性能.....	3
3. Demo 运行方法介绍	4
3.1 PC 端环境配置.....	4
3.1.1 adb 环境	4
3.1.2 python 及第三方库环境	4
3.2 模型转换.....	4
3.2.1 脚本介绍	4
3.2.2 转换并确认模型信息.....	4
3.3 启动 NNIDE 服务	4
3.3.1 Android	4
3.3.2 Linux.....	5
3.4 运行 demo	5
4. FAQ	6
4.1 调试手段.....	6
4.2 输入输出.....	6

1. NNIDE 简介

NNIDE (Neural Network Integrated Development Environment) 是由 Amlogic 开发的一套基于 nnEngine 引擎的综合性神经网络开发工具。旨在提供一个全面的平台，以支持在边缘计算设备上深度学习模型的部署。通过实现推理、可视化、精度分析和性能评估等关键功能，NNIDE 为 Amlogic 的客户提供了一个高效、便捷的工作流程，使客户能够更快速地评估其模型在 Amlogic NN 平台上的可行性和性能表现，缩短客户的整体开发周期。

1.1 NNIDE_tool 目录简介

NNIDE_TOOL 集成在工具中的路径：

```
adla-toolkit-binary-x.x.x.x/bin/adla Plug_in/tools/ NNIDE_tool
```

目录介绍：

```
1. The directory structure is as follows
demo
├── amlnn                                #Compiled file of nnide_tool
├── dataset                             #Dataset directory
│   ├── aml_dataset_crop_inception    #Classification model test data set(490 pictures)
│   └── input                          #Single frame test data
├── demo_adla_compare_buf.py           #Comparison script between the accuracy of the model file saved on the PC and the accuracy of the board end
├── demo_adla_jupyter.ipynb
├── demo_adla_run_datasets.py          #Classification model top1/top5 accuracy test script
├── demo_adla_run_multi_io.py         #Multi-input model inference script
├── demo_adla_run_single_io.py        #Single input model inference script
├── model_file                         #Test related model files
│   ├── adla                          #Converted adla file
│   └── original                      #Original model files, and conversion scripts
├── NNIDE_package                     #Board end IDE executable file
│   ├── android_32                   #Android 32-bit system IDE execution file
│   │   └── NNIDE
│   ├── android_64                   #Android 64-bit system IDE execution file
│   │   └── NNIDE
│   ├── linux_32                     #Linux 64-bit system IDE execution file
│   │   └── NNIDE
│   ├── yocto_64                     #Yocto 64-bit system IDE execution file
│   └── NNIDE
├── requirements.txt                  #PC environment dependency list
└── Readme.txt

Note:
1. Unzip command: tar -zxvf demo.tar.gz
```

2.NNIDE 功能

2.1 API 接口介绍

2.1.1 amlnn = AMLNN()

API	amlnn = engine.AMLNN(device_type='android', model_type='adla', model_path='./model_file/resnet18-v1-7_int8_A3_A311D2.adla', model_input_size='1 224 224 3', model_input_format='int8', output_type='fp32', run_cycles=1, need_hardware_info=True, need_runtime=True, need_bandwidth=True, need_visualize=True, loglevel='INFO')
功能	初始化 AMLNN 类
参数	device_type: 平台环境 (android, linux)
	model_type: 模型类型, 默认为 adla
	model_path: adla 模型的本地路径
	model_input_size: 模型的输入层的 shape, 排列方式为 n h w c, 如果是多输入模型, 用#分割
	model_input_format: 模型输入层的数据类型 (uint8/int8/int16/float32/int32/int64), 如果是多输入模型, 用#分割
	output_type: 希望获取的输出数据类型 (raw, fp32), 如果选择 raw, 那么会得到模型的原生输出 buf, 如果选 fp32, 那么内部会做反量化, 得到的输出数据类型均为 float32
	run_cycles: 推理次数, 可根据需求自行选择, 默认为 1
	need_hardware_info: 是否需要获取硬件信息 (平台名)
	need_runtime: 是否需要获取推理耗时
	need_bandwidth: 是否需要获取带宽信息
	loglevel: 日志级别 (DEBUG, INFO, WARNING, ERROR)
返回值	无

2.1.2 amlnn.init()

API	amlnn.init()
功能	模型初始化
参数	无
返回值	无

2.1.3 amlnn.set_input()

API	amlnn.init_runtime(input_data)
功能	设置模型输入
参数	input_data: 做好前处理之后的输入数据, 如果有多个输入, 那么需要先将它们的维度重塑为一维, 然后依次拼接
返回值	无

2.1.4 amlnn.inference()

API	output_data=amlnn.inference()
功能	模型推理并获取推理结果
参数	无
返回值	output_data: 模型输出结果

2.1.5 amlnn.destroy()

API	amlnn.destroy()
功能	卸载模型，释放相关资源
参数	无
返回值	无

2.1.6 amlnn.similarity_and_distance()

API	amlnn.similarity_and_distance(buf0, buf1)
功能	对比两组 buf 之间的相似度
参数	buf0, buf1: 参数中的 buf 是数组的形式，可通过 len(buf)来查看有多少个 buf，通过 buf[i].shape 来查看第 i 个子 buf 的维度
返回值	similarity, distance

2.2 模型推理

通过 NNIDE 提供的一系列接口，如 set_input 和 inference，客户能够直接在 PC 端使用 Python 脚本来执行特定操作用以完成量化模型在板端的推理任务并获取端侧的推理结果。这样做就能在测试阶段避免直接使用 NNSDK 接口，简化了端侧推理的复杂性，为客户提供了一个灵活且高效的工作流程。

在获得推理结果之后，客户还可以在同一个 Python 脚本中添加专门的后处理代码，对推理结果进行进一步的分析和处理，从而实现模型的具体应用功能。

```
... 模型初始化 ...
preprocess input_data
amlnn.set_input(input_data)
output_data = amlnn.inference()
postprocess output_data
... 模型释放 ...
```

2.3 精度对比

使用 NNIDE 提供的 similarity_and_distance 接口，客户可以快速对比模型转换中间过程保存的模型和模型板端运行结果的余弦相似度以及欧式距离，评估量化模型的精度

```
... 模型推理 ...
output_data = amlnn.inference()
output_data_original = customer_get_original_model_output
similarity, distance = amlnn.similarity_and_distance(output_data, output_data_original)
print(f'similarity: {similarity}')
print(f'distance: {distance}')
```

2.4 性能

在初始化 AMLNN 类时，将 need_runtime 和 need_bandwidth 参数设置为 True，loglevel 调整至 INFO 级别或更高，用户可以在模型推理完成后，获得关于模型运行的重要性能指标，包括 NPU 的总耗时和带宽使用情况。这些信息将通过命令行界面（cmd）直接打印出来，提供给用户实时的反馈。

```
amlnn = engine.AMLNN(..., need_runtime=True, need_bandwidth=True, loglevel='INFO', ...)
... 模型推理 ...
```

3. Demo 运行方法介绍

当前 NNIDE 支持运行在 Windows10 或 Ubuntu 20.04 操作系统和 Python3.8 环境下。

3.1 PC 端环境配置

3.1.1 adb 环境

PC 端和开发板通过 USB 接口连接，并确保 ADB 可正常识别到指定设备。

终端中执行命令：adb devices -l，可正常显示目标设备

执行命令：adb shell，确保 adb 功能正常

3.1.2 python 及第三方库环境

配置 python3.8 环境后，安装依赖库

```
pip install -r requirements.txt
```

3.2 模型转换

在 release 包的 demo/model_file/original 路径下放置了 demo 所对应的原始模型及相关脚本文件

3.2.1 脚本介绍

ide_model_convert.sh 是模型转换脚本，--model 选项用来指定原始模型路径，后续客户转换自己的模型时需要替换成自己模型的路径

check_input_param.sh 是模型信息确认脚本，\${adlinfo_release} -io 后面跟的是模型转换完之后生成的 adla 模型路径，同样的，客户需要替换成自己的 adla 模型路径

3.2.2 转换并确认模型信息

当前示例 demo 模型的路径已经指定好，客户只需要将整个 original 文件夹拷贝至工具包的 adla-toolkit-binary-x.x.x.x/demo 下，然后在命令行进入 original 目录，执行命令 ./ide_model_convert.sh 即可完成模型转换过程，生成的 adla 文件就在当前路径的 output 文件夹下

当前示例 demo 模型的路径已经指定好，在完成模型转换后，直接在命令行执行 ./check_input_param.sh 就能看到模型输入输出的相关信息

之后可以将这些生成的 adla 模型拷贝到 release 包的 demo/model_file/adla 路径下（目前已经在该路径下提供了一组转换好并加上了平台后缀的 adla 模型）

3.3 启动 NNIDE 服务

在 release 包的 demo/NNIDE_package 路径下放置了各不同环境下的 NNIDE 文件，客户可根据所用平台对应的环境选择相应的 NNIDE 文件

3.3.1 Android

打开终端，进入 release 包的 demo 目录，依次执行如下命令

```
adb root
```



```
adb remount
adb push your_path/NNIDE /vendor/bin (your_path 为客户选择的 NNIDE 所在路径)
adb shell
cd /vendor/bin
chmod +x NNIDE
./NNIDE
```

如果看到如下打印，说明 NNIDE 服务成功启动

```
nnEngine start to work, listening on port 8308
nnEngine start to work, listening on port 8309
nnEngine start to work, listening on port 8310
```

3.3.2 Linux

打开终端，进入 release 包的 demo 目录，依次执行如下命令

```
adb push your_path/NNIDE /usr/bin (your_path 为客户选择的 NNIDE 所在路径)
adb shell
cd /usr/bin
chmod +x NNIDE
./NNIDE
```

如果看到如下打印，说明 NNIDE 服务成功启动

```
nnEngine start to work, listening on port 8308
nnEngine start to work, listening on port 8309
nnEngine start to work, listening on port 8310
```

3.4 运行 demo

当前提供了 5 个示例 demo，分别是

单输入单输出模型	demo_adla_run_single_io.py
多输入多输出模型	demo_adla_run_multi_io.py
循环跑数据集	demo_adla_run_datasets.py
对比原始模型和量化模型输出 buf 差异	demo_adla_compare_buf.py
jupyter 环境 demo	demo_adla_jupyter.ipynb

对于单输入单输出模型，多输入多输出模型，循环跑数据集的 demo，在完成 3.1、3.2、3.3 之后，开启另一个终端，直接在命令行执行命令 `python xxx.py` 即可运行

对于对比原始模型和量化模型输出 buf 差异的 demo，由于原始模型是 tflite，所以还需要配置下能运行 tflite 的环境，`pip install tflite_runtime` 即可，然后也是在命令行执行命令 `python xxx.py` 来运行

对于运行在 jupyter 环境下的交互式 demo，需要先配置 jupyter 环境，`pip install jupyter` 即可，之后在命令行执行 `jupyter notebook` 即可进入网页版的 jupyter，在网页上打开 `demo_adla_jupyter.ipynb`，点击左上角的三角形来逐步执行代码块

4. FAQ

4.1 调试手段

Step 1. PC 端将日志级别设置为 DEBUG 等级, `loglevel='ERROR'`

Step 2. 板端将日志级别设置为 DEBUG 等级, `export NN_ENGINE_LOG_LEVEL=4`

Step 3. 再次执行 python 脚本

4.2 输入输出

输入：给到 `amlInn.set_input()` 接口的输入数据 `input_data` 是做好前处理的量化数据，并且格式需要是 nhwc 的，如果有多个输入，那么需要先将它们的维度重塑为一维，然后依次拼接。

输出：从 `amlInn.inference()` 接口获取到的 `output_data` 是 npu 推理之后的数据，它是一个数组，可通过 `len(output_data)` 来查看具体有几个输出 buf，通过 `output_data[i].shape` 来查看各个输出 buf 的 shape，buf 格式为 nhwc，客户可基于 python 端添加后处理代码验证模型精度是否正常